



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 09 - 2

NOMBRE COMPLETO: Monter González Luis Enrique

N.º de Cuenta: 318276605

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2025-1

FECHA DE ENTREGA LÍMITE: 27/10/2024

CALIFICACIÓN: _____

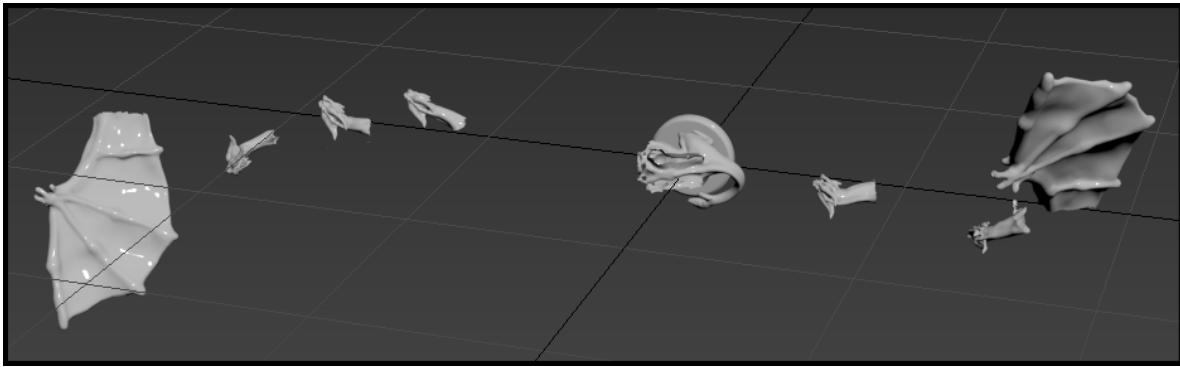
REPORTE DE PRÁCTICA:

Ejercicio 1: Hacer que en el arco que crearon se muestre la palabra: **PROYECTO CGEHC MONOPOLY**. animado desplazándose las letras de izquierda a derecha como si fuera letrero LCD/LED de forma cíclica.

-

Ejercicio 2: Separar las cabezas del Dragón y agregar las siguientes animaciones:

Se trabaja con el modelo incluido en la carpeta del proyecto y después de las separaciones se ve de la siguiente manera en 3ds Max.



Una vez ajustados los pivotes, se exportan y se incluyen en el proyecto.

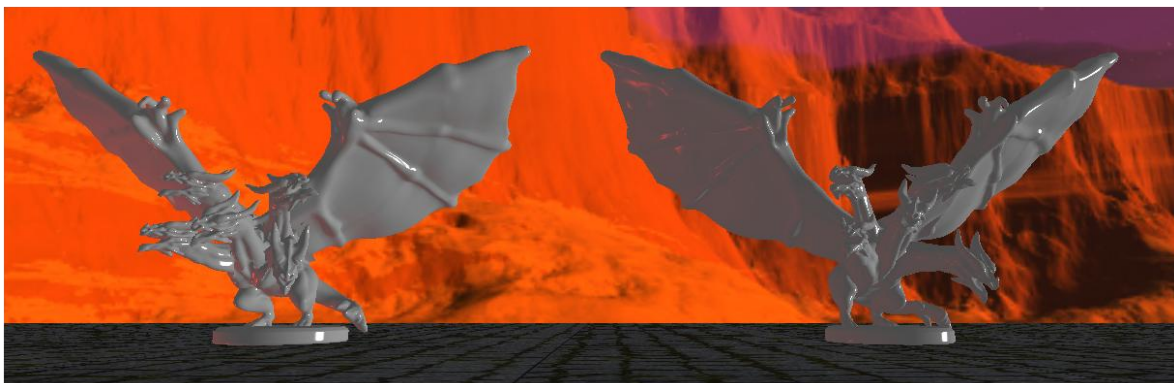
```
//Dragon  
Model CuerpoDragon;  
Model AlaIzquierda;  
Model AlaDerecha;  
Model Cabeza1;  
Model Cabeza2;  
Model Cabeza3;  
Model Cabeza4;  
Model Cabeza5;
```

```

//Dragon
CuerpoDragon = Model();
CuerpoDragon.LoadModel("Models/CuerpoDragon.obj");
AlaDerecha = Model();
AlaDerecha.LoadModel("Models/AlaDerecha.obj");
AlaIzquierda = Model();
AlaIzquierda.LoadModel("Models/AlaIzquierda.obj");
Cabeza1 = Model();
Cabeza1.LoadModel("Models/Cabeza1.obj");
Cabeza2 = Model();
Cabeza2.LoadModel("Models/Cabeza2.obj");
Cabeza3 = Model();
Cabeza3.LoadModel("Models/Cabeza3.obj");
Cabeza4 = Model();
Cabeza4.LoadModel("Models/Cabeza4.obj");
Cabeza5 = Model();
Cabeza5.LoadModel("Models/Cabeza5.obj");

```

Después de un acomodo exhaustivo manual, el dragón quedó de la siguiente manera.



El derecho es el modelo original y el izquierdo es el creado a partir de las partes separadas. El código es el siguiente.

```

//Cuerpo Dragon
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 5.0f, 6.0f));
//model = glm::translate(model, glm::vec3(0.0f-angulovaria/6, 5.0f+(3*sin(glm::radians(angulovaria*8))), 6.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
/*color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));*/
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CuerpoDragon.RenderModel();

```

```
//Ala Derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-0.25f, 1.0f, -0.3f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
AlaDerecha.RenderModel();

//Ala Izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-0.25f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
AlaIzquierda.RenderModel();
```

```
//Cabeza 1
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 1.0f, 0.15f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza1.RenderModel();
```

```
//Cabeza 2
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 1.1f, -0.2f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza2.RenderModel();
```

```
//Cabeza 3
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 0.8f, -0.3f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza3.RenderModel();
```

```
//Cabeza 4
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 0.85f, -0.1f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza4.RenderModel();
```

```
//Cabeza 5
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 0.85f, 0.2f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza5.RenderModel();
```

Movimiento del cuerpo ida y vuelta.

-

Aleteo

-

Cada cabeza se mueve de forma diferente de acuerdo con una función/algorithmo diferente (ejemplos: espiral de Arquímedes, movimiento senoidal, lemniscata, etc.)

Para el movimiento de las 5 cabezas se usaron las siguientes 5 funciones/algoritmos/fenómenos respectivamente: Función senoidal, Lemniscata, Órbita elíptica, Figura de Lissajous y Trayectoria de Bézier cuadrática.

Cabeza 1: La implementación fue igual que como se hizo con el dragón el día de la clase del laboratorio, cambiando los valores para que no se fuera muy lejos del cuerpo.

```
//Cabeza 1
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 1.0f + (0.15 * sin(glm::radians(angulovaria * 2))), 0.15f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza1.RenderModel();
```

Cabeza 2: La implementación de la Lemniscata se hace cumpliendo con la ecuación de Bernoulli en el plano 2D.

$$x = a \cdot \frac{\cos(\theta)}{1 + \sin^2(\theta)} \quad y = a \cdot \frac{\sin(\theta) \cdot \cos(\theta)}{1 + \sin^2(\theta)}$$

A es quien controla el tamaño de la lemniscata y el ángulo varia en el tiempo para generar el movimiento. El código es el siguiente.

```
//Cabeza 2
float a = 0.2f; // Escala de la lemniscata
float theta = glm::radians(angulovaria * 2); // Ángulo que varía para la animación

// Coordenadas en la lemniscata
float x = a * glm::cos(theta) / (1 + glm::sin(theta) * glm::sin(theta));
float y = a * glm::sin(theta) * glm::cos(theta) / (1 + glm::sin(theta) * glm::sin(theta));

model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f + x, 1.1f + y, -0.2f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza2.RenderModel();
```

Cabeza 3: La implementación de la órbita elíptica simula una trayectoria de ida y vuelta en forma ovalada alrededor de un punto central.



Su ecuación es la siguiente:

$$x = a \cdot \cos(\theta) \quad y = b \cdot \sin(\theta)$$

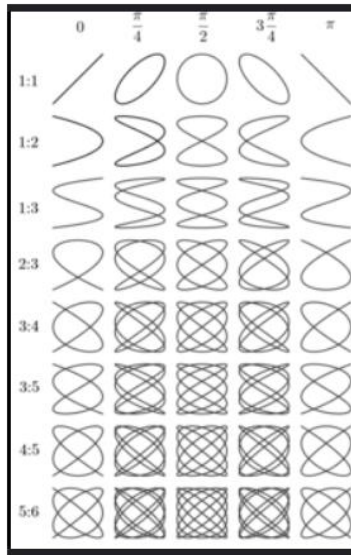
A y B son las longitudes de los ejes de la elipse. El código es el siguiente:

```
//Cabeza 3
float a2 = 0.2f; // Longitud del eje mayor
float b2 = 0.1f; // Longitud del eje menor
float theta2 = glm::radians(angulovaria);

float x2 = a2 * glm::cos(theta2);
float y2 = b2 * glm::sin(theta2);

model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 0.8f + y2, -0.3f + x2));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza3.RenderModel();
```

Cabeza 4: La implementación de la figura de Lissajous genera patrones que combinan movimientos senoidales con frecuencias distintas, dando como resultado trayectorias complejas.



Las formulas usadas fueron las siguientes:

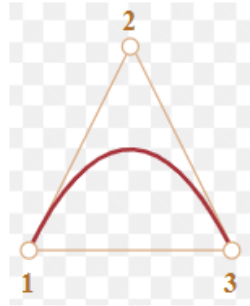
$$x = A \cdot \sin(f_x \cdot \theta + \delta) \quad y = B \cdot \sin(f_y \cdot \theta)$$

A y B son amplitudes y las “f” son las frecuencias. El código es el siguiente:

```
//Cabeza 4
float A = 0.1f;
float B = 0.05f;
float fx = 3.0f; // Frecuencia en x
float fy = 2.0f; // Frecuencia en y
float delta = glm::radians(45.0f); // Fase

float theta3 = glm::radians(angulovariable);
float x3 = A * glm::sin(fx * theta + delta);
float y3 = B * glm::sin(fy * theta);
model = modelaux;
model = glm::translate(model, glm::vec3(-0.6f, 0.85f + y3, -0.1f + x3));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza4.RenderModel();
```

Cabeza 5: La implementación de la trayectoria de Bézier Cuadrática define un punto de inicio, uno de control y un punto final. Esta curva da mucho control en la trayectoria y se puede usar para hacer movimientos tipo “rebote” entre los puntos.



La formulas son las siguientes:

$$x = (1 - t)^2 \cdot P0_x + 2 \cdot (1 - t) \cdot t \cdot P1_x + t^2 \cdot P2_x$$

$$y = (1 - t)^2 \cdot P0_y + 2 \cdot (1 - t) \cdot t \cdot P1_y + t^2 \cdot P2_y$$

T varia de 0 a 1 y representa el tiempo a lo largo de la curva y PN corresponde a los puntos. El código es el siguiente:

```
//Cabeza 5
glm::vec2 P0(-0.6f, 0.85f);
glm::vec2 P1(0.2f, 1.2f); // Punto de control
glm::vec2 P2(0.0f, 0.85f);

float t = 0.5f * (1 + glm::sin(glm::radians(angulovaria))); // Oscila entre 0 y 1

float x4 = (1 - t) * (1 - t) * P0.x + 2 * (1 - t) * t * P1.x + t * t * P2.x;
float y4 = (1 - t) * (1 - t) * P0.y + 2 * (1 - t) * t * P1.y + t * t * P2.y;

model = modelaux;
model = glm::translate(model, glm::vec3(x4, y4, 0.2f));
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cabeza5.RenderModel();
```

Cada cabeza debe de verse de un color diferente: roja, azul, verde, blanco, café.

Para implementar los colores se agregaron las siguientes líneas a las cabezas para mandar su color al shader.

Cabeza 1:

```
glm::vec3 color1 = glm::vec3(1.0f, 0.0f, 0.0f); // Rojo
glUniform3fv(uniformColor, 1, glm::value_ptr(color1));
```

Cabeza 2:

```
glm::vec3 color2 = glm::vec3(0.0f, 0.0f, 1.0f); // Azul  
glUniform3fv(uniformColor, 1, glm::value_ptr(color2));
```

Cabeza 3:

```
glm::vec3 color3 = glm::vec3(0.0f, 1.0f, 0.0f); // Verde  
glUniform3fv(uniformColor, 1, glm::value_ptr(color3));
```

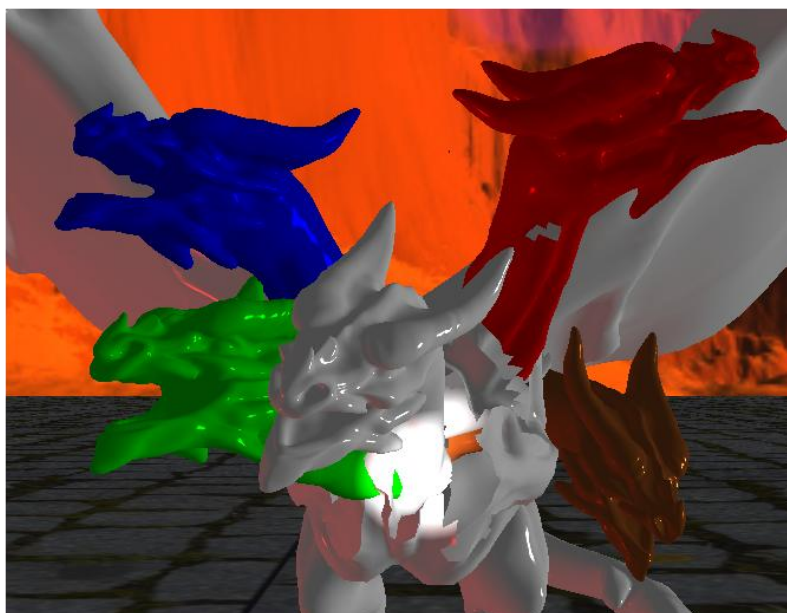
Cabeza 4:

```
glm::vec3 color4 = glm::vec3(1.0f, 1.0f, 1.0f); // Blanco  
glUniform3fv(uniformColor, 1, glm::value_ptr(color4));
```

Cabeza 5:

```
glm::vec3 color5 = glm::vec3(0.6f, 0.3f, 0.1f); // Café  
glUniform3fv(uniformColor, 1, glm::value_ptr(color5));
```

El resultado se ve de la siguiente manera.



Problemas

-

Conclusión

-

Bibliografía

• -